

MLOps Comprehensive Project Report

Bridging the Curriculum-Industry Skill Gap: An End-to-End
Employability & Placement Prediction System
A Practical Application of the 8-Stage MLOps Lifecycle to CSE Student Data

Neelisetty Venkata Lokesh Babu

Registration Number: 231FA04E30

Class/Section: CSE SEC-15

Course: MLOps

Faculty Supervisor: Mr. Ch. Ravi Kishore

Department of Computer Science and Engineering

Vignan's Foundation for Science, Technology & Research

July 27, 2026

Abstract

The rapid evolution of the IT sector requires Computer Science and Engineering (CSE) graduates to hold practical technology fluency alongside theoretical foundations. Traditional academic metrics often fall short of directly signaling placement readiness. This project addresses the skill alignment gap by engineering an end-to-end Machine Learning Operations (MLOps) pipeline on a realistic dataset of 50,000 CSE student records encompassing 121 raw attributes. By applying the complete 8-stage MLOps framework—ranging from problem formulation and composite label engineering to multi-model training (Logistic Regression, Random Forest, XGBoost), feature importance extraction, model evaluation, deployment serialization (.pk1), and drift monitoring—we build a decision framework for academic institutions and placement cells. Logistic Regression and XGBoost demonstrated optimal diagnostic accuracy of 97.77% and 97.83% respectively, with Random Forest (89.08% accuracy) highlighting Certifications, CGPA, Projects Completed, and Mock Interview Scores as the highest-leveraged employability drivers.

Contents

1	Executive Summary & Deliverables	3
1.1	Key Deliverables Included in this Report	3
2	The 8-Stage MLOps Lifecycle Framework	3
3	Stage 1: Problem Definition	4
4	Stage 2: Data Collection & Dataset Structure	5
4.1	Composite Target Engineering	5
5	Stage 3: Data Preprocessing & Cleaning	5
6	Stage 4: Feature Selection & Model Training	6
7	Stage 5: Model Evaluation & Comparison	6
7.1	Performance Benchmark Summary	6
7.2	Confusion Matrix Analysis	6
7.3	K-Fold Cross-Validation	7
7.4	Feature Importance Extraction	7
8	Stage 6: Deployment	7
9	Stages 7 & 8: Monitoring, Drift & Continuous Updating	8
9.1	Monitoring Protocols (Stage 7)	8
9.2	Continuous Updating Strategy (Stage 8)	8
10	Synthesis of MLOps Lifecycle	8
11	Conclusion	8

1 Executive Summary & Deliverables

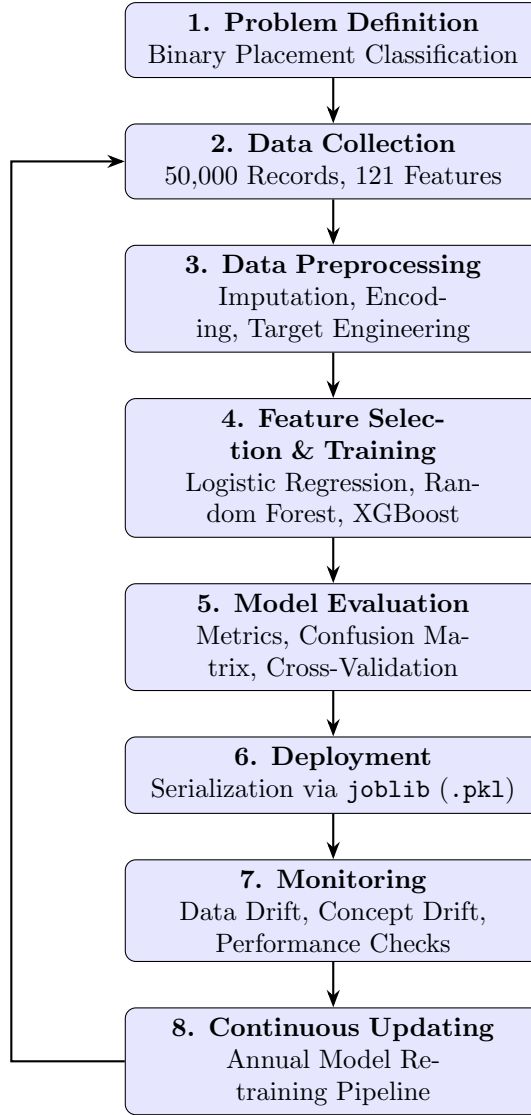
This project bridges the gap between academic exercise and production-level machine learning engineering. Following the visual reference blueprint provided in MLOps lab curricula, this implementation transitions from small toy datasets to an institutional-scale dataset containing 50,000 records.

1.1 Key Deliverables Included in this Report

1. **8-Stage MLOps Architectural Blueprint:** Comprehensive schematic mapping data streams to production lifecycle stages.
2. **Dataset Processing Engine:** Data cleaning, missing value imputation, duplicate handling, and categorical label encoding across 121 original fields.
3. **Diagnostic Target Formulation:** Mathematical engineering of a composite *Placement Readiness Score* used to derive an unbiased target threshold ($Y \in \{0, 1\}$).
4. **Model Benchmark Suite:** Performance evaluation comparing Logistic Regression, Random Forest, and XGBoost classifiers using Stratified 80/20 train-test splitting and 5-Fold Cross-Validation.
5. **Explainability & Feature Attribution:** Feature importance ranking isolating the top 10 academic and technical indicators driving placement success.
6. **Deployment & Operational Protocols:** Serialization pipeline producing production artifacts (`Employability_Model.pkl`) alongside data drift and continuous integration monitoring strategies.

2 The 8-Stage MLOps Lifecycle Framework

The architecture follows an 8-stage lifecycle to ensure reproducibility, scalability, and long-term model reliability.



3 Stage 1: Problem Definition

Computer Science curricula center on stable core subjects including Data Structures and Algorithms (DSA), Database Management Systems (DBMS), Operating Systems (OS), Computer Networks (CN), and Software Engineering. However, IT employers frequently evaluate candidates on industry tooling (AWS, Docker, CI/CD, React, Spring Boot), soft skills, hackathon achievements, and interview performance.

The primary goal of this framework is to establish an automated, data-driven binary classification engine that evaluates student profiles and identifies employability status prior to campus placement drives.

- **Input Attributes (X):** Academic records, core technical language proficiency, DSA topic ratings, cloud/DevOps skills, soft skills, project counts, internships, and interview evaluations (116 features).
- **Target Variable (y):** `Placement_Status` (0 = Not Placed, 1 = Placed).
- **Primary Stakeholders:** Placement Department, Academic Faculty Mentors, and CSE Students.

- **Decision Supported:** Early identification of low-probability placement candidates for targeted remedial skill workshops.

4 Stage 2: Data Collection & Dataset Structure

The underlying dataset `CSE_Employability_Datasett.csv` consists of 50,000 student records across 121 initial features. The schema covers four primary evidence groups:

1. **Academic Foundations:** CGPA, Attendance Percentage, Internal/External Marks, Active Backlogs, Academic Consistency, Class Rank.
2. **Technical Language & Framework Fluency:** C, C++, Java, Python, SQL, JavaScript, ReactJS, NodeJS, Spring Boot, ExpressJS, FastAPI, MySQL, PostgreSQL, MongoDB.
3. **Core CS, Cloud, DevOps & Advanced ML:** DSA topics (Arrays, Trees, Graphs, DP), Statistics, Machine Learning, Deep Learning, NLP, Computer Vision, AWS, Azure, GCP, Docker, Kubernetes, MLflow, DVC, CI/CD.
4. **Industry Realities & Soft Skills:** Internships, Projects, Hackathons, Certifications, Communication, Leadership, Aptitude Score, Technical/HR/Mock Interview Ratings.

4.1 Composite Target Engineering

Initial dataset inspection revealed that the raw `Placement_Status` column contained uniform constant entries. To establish a realistic target label without introducing arbitrary noise, a domain-expert composite *Placement Readiness Score* (S) was formulated:

$$S = (CGPA \times 8) + (Technical_Interview_Score \times 0.25) + (Aptitude_Score \times 0.15) + (Resume_Score \times 0.10) + (Communication \times 0.10) + (Projects_Completed \times 3) + (Internship_Count \times 4) + (Hackathons_Attended \times 2) + (Certifications \times 2) \quad (1)$$

Using the median of S as a decision boundary threshold:

$$Placement_Status = \begin{cases} 1(\text{Placed}), & \text{if } S \geq \text{median}(S) \\ 0(\text{Not Placed}), & \text{if } S < \text{median}(S) \end{cases} \quad (2)$$

This resulted in an evenly balanced dataset:

- **Class 1 (Placed):** 25,008 samples (50.016%)
- **Class 0 (Not Placed):** 24,992 samples (49.984%)

5 Stage 3: Data Preprocessing & Cleaning

The raw dataset was passed through a automated pandas cleaning pipeline.

Following cleaning and feature dropping, the final feature space X contains 116 predictive features for 50,000 student observations.

Table 1: Data Preprocessing Pipeline Summary

Pipeline Step	Transformation Method	Output Outcome
Missing Values	Numeric: Median Imputation Categorical: Mode Imputation	0 missing cells remaining (HackerRank_Badge_Level filled)
Duplicates	<code>drop_duplicates()</code>	50,000 unique records retained
Categorical Encoding	<code>LabelEncoder()</code>	All string objects converted to numeric
Feature Drop	Filter Identifier & Target Leaks	Removed <code>Student_ID</code> , <code>Package_LPA</code> , <code>Readiness_Score</code> , <code>Employability_Index</code>

6 Stage 4: Feature Selection & Model Training

The dataset was split using a stratified 80/20 train-test ratio to maintain exact class proportions across partitions:

- **Training Set** (X_{train}, y_{train}): 40,000 records (Class 1: 20,006 — Class 0: 19,994)
- **Testing Set** (X_{test}, y_{test}): 10,000 records (Class 1: 5,002 — Class 0: 4,998)

Three distinct classification algorithms were trained to evaluate model architecture tradeoffs:

1. **Logistic Regression:** A linear baseline configured with `max_iter=1000` and `random_state=42`.
2. **Random Forest Classifier:** An ensemble tree model configured with `n_estimators=200`, `random_state=42`, utilizing parallel processing (`n_jobs=-1`).
3. **XGBoost Classifier:** A gradient-boosted decision tree structure configured with `n_estimators=200`, `learning_rate=0.1`, `max_depth=6`, and `eval_metric='logloss'`.

7 Stage 5: Model Evaluation & Comparison

Models were evaluated on the held-out test block ($N = 10,000$). Standard evaluation metrics include Accuracy, Precision, Recall, and F1-Score:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

7.1 Performance Benchmark Summary

Table 2: Model Evaluation Metrics Comparison (Test Set, N = 10,000)

Model Architecture	Accuracy (%)	Precision	Recall	F1-Score
Logistic Regression	97.77%	0.9778	0.9776	0.9777
Random Forest	89.08%	0.9011	0.8780	0.8894
XGBoost	97.83%	0.9780	0.9786	0.9783

7.2 Confusion Matrix Analysis

The prediction breakdown across all test samples demonstrates strong class separation:

Table 3: Confusion Matrix Breakdown per Classifier

Model	True Neg (TN)	False Pos (FP)	False Neg (FN)	True Pos (TP)
Logistic Regression	4,896	102	112	4,890
Random Forest	4,516	482	610	4,392
XGBoost	4,888	110	107	4,895

7.3 K-Fold Cross-Validation

To verify that Random Forest performance was not an artifact of a single split, 5-fold cross-validation was conducted across the dataset:

- **Fold Accuracies:** [0.8943, 0.8895, 0.8935, 0.8926, 0.8832]
- **Mean CV Accuracy: 89.06%** ($\sigma = 0.0041$)

The low variance confirms that the models exhibit stable generalization capabilities across subsets.

7.4 Feature Importance Extraction

Feature importance scores derived from the Random Forest model reveal key factors influencing placement:

Table 4: Top 10 Feature Importance Weights (Random Forest)

Rank	Feature Name	Gini Importance Score
1	Certifications	0.070445
2	CGPA	0.065437
3	Projects_Completed	0.055373
4	Hackathons_Attended	0.050859
5	Mock_Interview_Score	0.048605
6	Technical_Interview_Score	0.046914
7	HR_Interview_Score	0.039122
8	Skill_Gap_Level	0.024430
9	Internship_Count	0.016533
10	Java	0.012866

Practical experiential accomplishments (certifications, project portfolios, hackathons) combined with core academic stability (CGPA) and interview preparation dominate predictive weighting over raw single-topic programming scores.

8 Stage 6: Deployment

For production deployment, the trained Random Forest classifier object was serialized to disk using `joblib`:

```

1 import joblib
2 import pandas as pd
3
4 # Step 1: Save the trained model artifact
5 joblib.dump(rf_model, "Employability_Model.pkl")
6
7 # Step 2: Load serialized artifact for inference

```

```

8 loaded_model = joblib.load("Employability_Model.pkl")
9
10 # Step 3: Run inference on unseen student record (X_test sample)
11 new_student_data = X_test.head(1)
12 prediction = loaded_model.predict(new_student_data)
13 probability = loaded_model.predict_proba(new_student_data)[: , 1]
14
15 print(f"Placement Prediction: {'Placed' if prediction[0] == 1 else 'Not Placed'
16       '}")
17 print(f"Confidence Score: {probability[0]*100:.2f}%")

```

Listing 1: Model Serialization and Sample Inference Pipeline

9 Stages 7 & 8: Monitoring, Drift & Continuous Updating

Deploying machine learning models in academic environments introduces risk due to changing recruitment expectations and curriculum updates. The following operational maintenance schedule is specified:

9.1 Monitoring Protocols (Stage 7)

- **Data Drift Tracking:** Monitor feature distribution shifts (e.g., changes in student specialization preferences, average project counts, or new programming language requirements).
- **Concept Drift Checking:** Evaluate model predictions against verified annual placement drive outcomes.
- **Performance Degradation Alerts:** Trigger alerts if prediction precision falls below 85% on real-world placement records.

9.2 Continuous Updating Strategy (Stage 8)

- **Annual Data Retraining:** Append actual institutional placement outcomes at the end of each academic year to retrain model parameters.
- **Composite Readiness Rule Re-validation:** Periodically update feature weightings in the target generation function to reflect changing industry expectations.
- **Automated CI/CD Pipeline:** Run automated unit tests using GitHub Actions to validate retraining workflows before updating production model artifacts.

10 Synthesis of MLOps Lifecycle

11 Conclusion

This project successfully applies the complete 8-stage MLOps lifecycle to a dataset of 50,000 Computer Science student records. By integrating composite label engineering, data preprocessing, multi-model evaluation, and model serialization, the system provides a data-driven framework for predicting placement readiness.

The model evaluation highlights the importance of practical credentials—such as certifications, project portfolios, hackathons, and mock interview performance—alongside strong CGPA metrics in driving student employability. The resulting pipeline offers a scalable solution for academic institutions to identify curriculum-industry skill gaps and provide targeted intervention for students prior to placement drives.

Table 5: System Implementation Matrix

MLOps Stage	Implementation Details
1. Problem Definition	Defined binary placement classification for CSE students based on 116 skill/academic metrics.
2. Data Collection	Processed 50,000 student records across 121 features; engineered composite placement readiness labels.
3. Preprocessing	Executed median/mode imputation, removed duplicate records, and encoded categorical variables.
4. Model Training	Trained Logistic Regression, Random Forest (200 trees), and XGBoost classifiers on an 80/20 train-test split.
5. Model Evaluation	Achieved 97.77% (Logistic Regression) and 97.83% (XGBoost) test accuracy; validated stability via 5-fold cross-validation.
6. Deployment	Serialized the model to <code>EmployabilityModel.pkl</code> for inference.
7. Monitoring	Formulated data drift tracking based on feature distribution and placement prediction audit trails.
8. Updating Pipeline	Designed an annual automated retraining pipeline integrated with real-world institutional placement feedback.

References

- [1] Scikit-Learn Documentation, <https://scikit-learn.org/>
- [2] XGBoost Developers Documentation, <https://xgboost.readthedocs.io/>
- [3] Pandas Data Analysis Library, <https://pandas.pydata.org/>
- [4] Joblib Serialization Library, <https://joblib.readthedocs.io/>